STANCE: Learning Agent Report

ME5418 Group 42 · Vishal Mangla, Mukul Yadav, Muthaiah Mahadevan

1 Overview

Report 2 described the environment, the two networks (an MLP and an LSTM, both trained with PPO) and the evaluation. This report covers what the agent stage adds. A single demo now loads the shipped weights, or trains a fresh network, and compares the result with the best fixed action on identical episodes. The demo can also open the new MuJoCo viewer, which plays a few episodes of the policy (5 by default, up to 10) on the deforming ground so that the user can watch the behaviour and not only read the numbers. On the shipped seed-0 weights, the LSTM reaches a return of 89.76 against 87.90 for the best fixed action, at almost the same survival. For us, the most instructive outcome lies in the reward function. To judge where exactly our agent can gain, we re-scored a grid of 45 constant actions under different reward weightings. Heavier impact penalties resulted in the agent learning to fall as its best strategy, and hence we have kept the original reward.

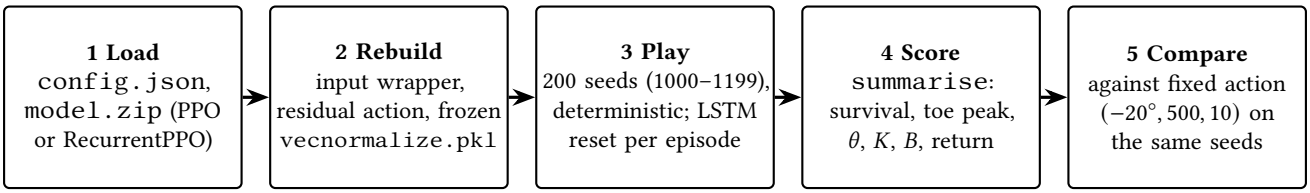
2 Code explanation

We organised the code to let the reader follow one path from a saved run to a printed comparison. The training, residual-action and network files (`agent/`, `common/` and both `network.py`) are identical to Report 2. Only `common/evaluate.py` differs: it is now the full version, while Report 2 contained a trimmed one with the same scoring functions. The results are therefore unchanged, and the demo calls the same code that produced them.

Path	Responsibility (new in this submission)	Author
`demo_agent.py`	Loads a saved run, or trains one, and evaluates it against the best fixed action on the same seeds. With `--show`, it can also open the MuJoCo viewer to play 5 episodes by default (up to 10).	All
`trained_networks/`	Both networks, seeds 0–2: weights, normalisation statistics, config, learning curve, and per-episode results on both grounds.	All

2.1 How the demo works

Running `python demo_agent.py --model <run folder> --episodes 200` follows the five stages below. Stage 2 deserves a comment. A policy behaves as it did in training only if its observations are scaled with the *training* statistics. We therefore load the normaliser with updating switched off, which freezes its running mean and variance at the values learned during training and stops the demo episodes from changing them. For anyone running the demo, the printed numbers therefore stay comparable with Report 2.



The demo takes about 30 s per network (on pre-trained model mode). The LSTM stays close to the fixed action, while the MLP moves further away from it.

Training mode runs the same pipeline from scratch. The command `python demo_agent.py --train --steps 4096` builds 8 parallel environments, trains the MLP (or the LSTM with `--network lstm`) for 4096 steps, saves the run files and evaluates the result as above. The 8 environments match the setup of our full training runs in Report 2. Each update collects 256 steps from each of them, which gives 2048 samples, and 4096 steps therefore amount to only two updates. This is far too short for learning. It does confirm that rollout, update, saving and evaluation work together. A full 300k-step run takes about 6.8 min for the MLP and about 17 min for the LSTM, which are the times reported for Report 2.

3 Existing codes and libraries

The packages and libraries used are identical to our previous submission.

Library (version)	Used for	What we did to make it work
PyTorch 2.14.1	Layers, autograd, Adam.	Wrote both network bodies (`HistoryMLP`, `FrameEncoder`) as feature extractors.
Stable-Baselines3 2.9.0	PPO, actor–critic policy, `VecNormalize`.	Plugged our bodies in through `policy_kwargs`, and saved the normalisation statistics with the weights to let the demo reproduce training.
sb3-contrib 2.9.0	`RecurrentPPO` (LSTM).	Carried the hidden state between steps and reset it each episode, in training and in our evaluation adapter.
Gymnasium 1.4.0, MuJoCo 3.15.0	Spaces and wrappers, and physics (Report 1).	Wrote the input and residual-action wrappers, which left the environment code untouched.

4 Reflections and lessons learned

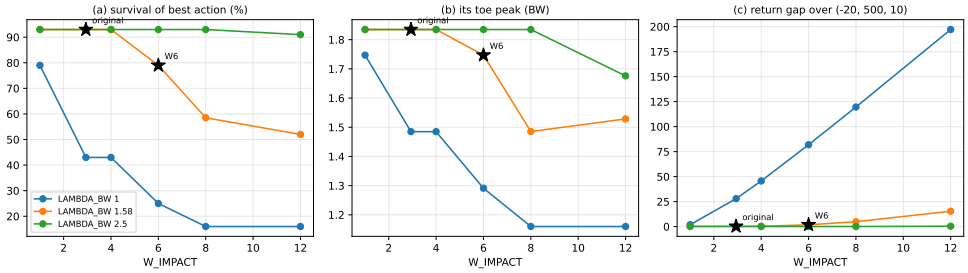
4.1 Changes since Report 2

Our previous report noted five future additions.

Report 2 said we would…	Status	What happened
Add the final weights	**Done**	Both networks, three seeds each.
Test a larger impact penalty (weight 2.94 → 6)	**Done**	Run as a sweep (§4.2), with the reward unchanged.
Add a rollout video and fix the MuJoCo `render()`	**Partial**	The MuJoCo viewer plays the episodes as intended, but the process can hang or crash at exit on some machines (a GLFW issue). The render and the recorder still need work.
Run a final benchmark (more seeds, proposal metrics)	***Moved***	Report 2's three seeds are re-used for now.
Widen the training ground and add a fresh out-of-range set	***Moved***	Postponed to the final version.

4.2 What the reward weights do

The reward penalises the squared toe-force excess above a threshold of $\lambda_{\mathrm{BW}} = 1.58$ body weights, scaled by an impact weight $W_{\mathrm{impact}} = 2.94$. We re-ran the 45 constant actions, kept their per-step toe forces in memory (nothing is saved), and recomputed the return of every action without retraining for 18 combinations of impact weight (1 to 12) and threshold (1, 1.58 and 2.5 body weights), and recorded the best action for each. We call the setting with impact weight 6 *W6*. It leaves a return headroom for the agent to optimise. The figure is shipped in `figures/`, but the sweep and plotting scripts are not part of this submission.



Best fixed action per weighting (training ground): (a) survival, (b) toe peak, (c) return gap to $(-20, 500, 10)$. Stars = the original weights and W6.

As the impact penalty grows, the best constant action trades survival for lower impact. At the original weights, the best action $(-20, 500, 6)$ keeps 93.0% of episodes alive. With a threshold of 1.58 body weights, raising the impact weight from 2.94 to 6 (W6) lowers its survival to 79.0%, and raising it to 12 lowers it to 52.0%. The penalty lowers the peak toe force only through softer actions, which fall more often (panel a). At the original weights, our reference action $(-20, 500, 10)$ trails the best constant action by only 0.26 return, and hence the agent has little room to improve.

4.3 What we learned

(1) **The weights define the task:** beyond a certain impact weight, the best-scoring behaviour is to fall softly instead of standing. A heavier impact penalty asks for a different behaviour (minimising impact) and not for a harder version of the original one (standing with low impact).

(2) **We still keep the original reward:** We chose W6 after viewing the sweep results. Choosing a setting after seeing how the sweep turns out can bias the comparison in its favour, because we picked the setting that left room for the agent to improve. The W6 returns are therefore not comparable with the data of Report 2, which was produced under the original reward without such a choice. The allowed action offset also restricts the agent to damping between about 4.1 and 10 N m s/rad, and the best action may therefore lie outside its reach. W6 remains an exploratory extension.

4.4 Towards the final version (20 Nov)

Four pieces of work remain.

- **Benchmark and reproducibility:** more seeds, the remaining proposal metrics (mean ± std, fall type and peak force against stiffness), and the sweep and plotting scripts so that the figure can be regenerated.
- **MuJoCo render and rollout video:** improve the MuJoCo render, and write and test the rollout-video code.
- **Generalisation:** test on a wider training ground and a fresh out-of-range set, with a larger seed pool.
- **Additional visual elements:** improve understanding and clarity, for example with an arrow that grows with the toe-force impact or with another return signal.

AI-use declaration
We used Claude (Anthropic) to draft a template for the code files that kept settings and keys compatible across team members, to help debug code and flag merge discrepancies, and to generate the plotting code for this report. The final code was written by us, and we are responsible for its content and quality.